

1. 前端需要实现的功能

1.1 BPMN配置界面改造

前端需要在BPMN配置界面中添加以下配置项：

```
// 文本内容提取节点配置界面
{
  "nodeType": "textExtract",
  "config": {
    "inputText": {
      "label": "选择要提取内容的文本变量",
      "type": "variableSelector",
      "value": "${mutileFile}"
    },
    "model": {
      "label": "选择模型",
      "type": "modelSelector",
      "value": "deepseek-v3.1"
    },
    "extractionRequirement": {
      "label": "提取要求描述",
      "type": "textarea",
      "placeholder": "根据用户的描述提取",
      "value": "${sys_query}"
    },
    "outputVariableName": {
      "label": "输出变量名",
      "type": "input",
      "value": "fileResult"
    }
  }
}
```

1.2 变量引用支持

前端需要支持变量引用功能，允许用户在配置中使用 `${variableName}` 格式引用其他节点的输出。

1.3 大模型参数配置

前端需要提供大模型参数配置界面，包括：

- 模型选择（如deepseek-v3.1）
- API密钥配置（可选，如果需要的话）
- 提示词配置（提取要求描述）

1.4 输出变量配置

前端需要允许用户配置输出变量名，以便后续节点可以引用该变量。

2. 联调测试步骤

2.1 准备测试数据

1. 创建一个包含文本内容的测试文件
2. 配置BPMN流程，将文本内容作为输入变量
3. 设置提取要求描述

2.2 执行测试

1. 启动工作流引擎
2. 触发工作流执行
3. 监控日志，确保节点正常执行
4. 验证输出变量是否正确设置

2.3 验证结果

1. 检查输出变量 `fileResult` 是否包含提取的内容
2. 验证提取内容是否符合提取要求描述
3. 确保大模型参数正确传递

3. 前端需要做的具体修改

3.1 修改节点配置组件

在前端的节点配置组件中，添加对新参数的支持：

```
// 文本内容提取节点配置组件
export default {
  name: 'TextExtractConfig',
  props: ['node'],
  data() {
    return {
      config: {
        inputText: this.node.config.inputText || '',
        model: this.node.config.model || '',
        extractionRequirement: this.node.config.extractionRequirement || '',
        outputVariableName: this.node.config.outputVariableName || 'fileResult'
      }
    };
  },
  methods: {
    saveConfig() {
      // 保存配置到BPMN XML
      this.$emit('update:node', {
        ...this.node,
        config: this.config
      });
    }
  }
};
```

3.2 修改BPMN生成器

在BPMN生成器中，添加对新参数的支持：

```
// BPMN生成器 - 文本内容提取节点
function generateTextExtractNode(node) {
  return `
    <bpmn:serviceTask id="${node.id}" name="${node.name}">
      <bpmn:extensionElements>
        <camunda:properties>
          <camunda:property name="llmConfig" value='{ "provider": "deepseek",
"model": "${node.config.model}", "apiKey": "your-api-key", "apiBase":
"https://api.deepseek.com"}' />
          <camunda:property name="inputText" value="${node.config.inputText}" />
          <camunda:property name="extractionRequirement"
value="${node.config.extractionRequirement}" />
          <camunda:property name="outputVariableName"
value="${node.config.outputVariableName}" />
        </camunda:properties>

        <camunda:inputOutput>
          <camunda:inputParameter name="inputText">
            <camunda:expression>${node.config.inputText}</camunda:expression>
          </camunda:inputParameter>
          <camunda:inputParameter name="extractionRequirement">
            <camunda:expression>${node.config.extractionRequirement}</camunda:expression>
          </camunda:inputParameter>

          <camunda:outputParameter name="${node.config.outputVariableName}">
            <camunda:expression>${extractedContent}</camunda:expression>
          </camunda:outputParameter>
        </camunda:inputOutput>
      </bpmn:extensionElements>

      <bpmn:implementation
type="class">com.yundingtech.agent.work.modules.workflow.delegate.ContentExtract
Service</bpmn:implementation>
    </bpmn:serviceTask>
  `;
}
```

3.3 添加变量选择器

前端需要实现一个变量选择器组件，允许用户从已有的变量中选择输入变量：

```
// 变量选择器组件
export default {
  name: 'VariableSelector',
  props: ['variables', 'value'],
  data() {
    return {
      selected: this.value
    };
  },
  methods: {
    onChange(value) {
      this.selected = value;
      this.$emit('input', value);
    }
  }
}
```

```
};
```

4. 联调测试建议

1. **先测试简单场景**：使用简单的提取要求描述，如"提取所有数字"
2. **逐步增加复杂度**：测试更复杂的提取要求，如正则表达式匹配
3. **验证变量传递**：确保输出变量可以在后续节点中正确引用
4. **检查错误处理**：测试输入为空或提取要求描述为空的情况

完成以上前端功能后，您就可以与后端进行联调测试了。